University Echahid Hama Lakhdar, El Oued
Institute of Exact Sciences | Department of Computer Science
Master 2: Artificial Intelligence and Data Science

LAB 02 | Object Detection Using Original R-CNN

Computer Vision · Google Colab (GPU recommended) · Python · Guided practical work

Objective: Implement the R-CNN pipeline step by step: region proposals, CNN features, SVM classification, NMS and detection evaluation.

Note: This is a teaching implementation using ResNet18 and a small SVM; learned bounding-box regression is not included.

TASK 0 Google Colab Setup

Open Google Colab → **Runtime → Change runtime type → GPU** (if available). Run each cell **in order**. The first cell installs the extra packages; restart the runtime only if Colab explicitly requests it. Downloading COCO128 and pretrained weights requires internet access. If the GPU is unavailable, reduce **MAX_IMAGES** and **MAX_PROPOSALS**.

CODE 01 — Install requirements (Colab)

CODE 01 / PYTHON — GOOGLE COLAB

```python
import sys, subprocess
subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q',
    'opencv-contrib-python-headless>=4.8,<5', 'scikit-learn>=1.3',
    'matplotlib>=3.7', 'pandas>=2', 'tqdm>=4.66'])
import torch, torchvision, cv2, numpy as np, matplotlib.pyplot as plt
print('PyTorch:', torch.__version__, '| TorchVision:', torchvision.__version__)
print('OpenCV:', cv2.__version__)
print('GPU available:', torch.cuda.is_available())
assert hasattr(cv2, 'ximgproc') and hasattr(cv2.ximgproc, 'segmentation'),     'Selective
    Search unavailable: restart runtime after installing opencv-contrib.'
```

TASK 1 Dataset Preparation (COCO Subset)

The reference handout uses COCO. To keep downloads feasible on Colab, we use **COCO128** rather than the complete COCO 2017 dataset. Its text annotations contain **(class_id, x_center, y_center, width, height)** in normalized coordinates. This task converts annotations to pixel-space bounding boxes and splits images by image, preventing the same image from appearing in training and testing.

CODE 02 — Download COCO128, select classes and examine annotations

CODE 02 / PYTHON — GOOGLE COLAB

```python
from pathlib import Path
from collections import Counter
from sklearn.model_selection import train_test_split
from tqdm.auto import tqdm
import urllib.request, zipfile, random

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
ROOT = Path('/content/rcnn_lab'); ROOT.mkdir(exist_ok=True, parents=True)
URL = 'https://github.com/ultralytics/assets/releases/download/v0.0.0/coco128.zip'
archive = ROOT / 'coco128.zip'
```

CODE 02 / PYTHON / CONTINUED GOOGLE COLAB

```python
if not (ROOT / 'coco128' / 'images' / 'train2017').exists():
    if not archive.exists():
        urllib.request.urlretrieve(URL, archive)
    with zipfile.ZipFile(archive) as z:
        z.extractall(ROOT)
IMG_DIR = ROOT / 'coco128' / 'images' / 'train2017'
LBL_DIR = ROOT / 'coco128' / 'labels' / 'train2017'
# Original COCO class indices: 0=person, 2=car, 16=dog
COCO_CLASSES = {0: 'person', 2: 'car', 16: 'dog'}

def load_record(path):
    image = cv2.imread(str(path))
    if image is None: return None
    h, w = image.shape[:2]
    boxes = []
    label_path = LBL_DIR / (path.stem + '.txt')
    if label_path.exists():
        for line in label_path.read_text().splitlines():
            vals = line.strip().split()
            if len(vals) != 5: continue
            cls = int(float(vals[0]))
            if cls not in COCO_CLASSES: continue
            xc, yc, bw, bh = map(float, vals[1:])
            x1 = max(0, (xc - bw/2) * w); y1 = max(0, (yc - bh/2) * h)
            x2 = min(w, (xc + bw/2) * w); y2 = min(h, (yc + bh/2) * h)
            if x2 > x1 and y2 > y1:
                boxes.append((np.array([x1,y1,x2,y2], dtype=np.float32), cls))
    return {'path':path, 'boxes':boxes}

records = [load_record(p) for p in sorted(IMG_DIR.glob('*')) if p.suffix.lower() in
    ('.jpg','.jpeg','.png')]
records = [r for r in records if r is not None and r['boxes']]
print('Images with selected classes:', len(records))
print('Object counts:', dict(Counter(c for r in records for _,c in r['boxes'])))
assert len(records) >= 10, 'Not enough labeled images. Check extraction and labels.'
train_records, test_records = train_test_split(records, test_size=0.25, random_state=SEED)
MAX_IMAGES = 40  # lower to 16 for a shorter session
train_records = train_records[:MAX_IMAGES]
test_records = test_records[:max(6, MAX_IMAGES//4)]
print('Train images:', len(train_records), '| Test images:', len(test_records))
```

CODE 03 — Inspect image and its ground-truth boxes

CODE 03 / PYTHON GOOGLE COLAB

```python
from matplotlib.patches import Rectangle
r = train_records[0]
im = cv2.cvtColor(cv2.imread(str(r['path'])), cv2.COLOR_BGR2RGB)
fig, ax = plt.subplots(figsize=(9,6)); ax.imshow(im)
for box, cls in r['boxes']:
    x1,y1,x2,y2 = box
    ax.add_patch(Rectangle((x1,y1),x2-x1,y2-y1,fill=False,edgecolor='lime',linewidth=2))
    ax.text(x1,y1,COCO_CLASSES[cls],color='yellow',fontsize=10)
ax.set_title('Authentic COCO ground-truth boxes'); ax.axis('off'); plt.show()
```

Checkpoint 1 — Dataset

Check: Image displays with correctly placed COCO labels · Training and test image sets are disjoint · Per-class object counts have been recorded

01. Why do we split by image rather than by region proposal?
02. Why must bounding-box coordinates change when an image is resized?
03. What limitations arise from selecting only a small subset of COCO?

TASK 2 Region Proposals with Selective Search

Selective Search proposes candidate regions **without learning a detector**. It typically generates many more boxes than there are objects. We resize long image edges only for computational efficiency, while keeping every proposal in the resized coordinate system. To simplify the remaining pipeline, we instead use a fixed **384×384** resized representation and scale COCO boxes accordingly.

CODE 04 — Generate and visualize Selective Search regions

CODE 04 / PYTHON GOOGLE COLAB

```python
SIZE = 384
MAX_PROPOSALS = 120  # use 250+ for a more complete but slower experiment

def prepare_image(record):
    raw = cv2.cvtColor(cv2.imread(str(record['path'])), cv2.COLOR_BGR2RGB)
    h,w = raw.shape[:2]
    resized = cv2.resize(raw, (SIZE,SIZE))
    gt = [(np.array([b[0]*SIZE/w,b[1]*SIZE/h,b[2]*SIZE/w,b[3]*SIZE/h],dtype=np.float32), c)
          for b,c in record['boxes']]
    return resized, gt

def selective_proposals(image, limit=MAX_PROPOSALS):
    ss = cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()
    ss.setBaseImage(cv2.cvtColor(image, cv2.COLOR_RGB2BGR))
    ss.switchToSelectiveSearchFast()
    rects = ss.process()  # (x,y,w,h)
    boxes=[]; seen=set()
    for x,y,w,h in rects:
        if w*h < 32*32 or w < 20 or h < 20: continue
        box=(int(x),int(y),int(min(x+w,SIZE)),int(min(y+h,SIZE)))
        if box not in seen and box[2]>box[0] and box[3]>box[1]:
            seen.add(box); boxes.append(box)
        if len(boxes)>=limit: break
    return np.array(boxes,dtype=np.float32).reshape(-1,4)

sample, gt = prepare_image(train_records[0])
proposals = selective_proposals(sample)
fig, ax = plt.subplots(figsize=(7,7)); ax.imshow(sample)
for x1,y1,x2,y2 in proposals[:40]:
    ax.add_patch(Rectangle((x1,y1),x2-x1,y2-y1,fill=False,edgecolor='red',linewidth=.7))
ax.set_title(f'{len(proposals)} proposals; first 40 displayed'); ax.axis('off'); plt.show()
```

CODE 05 — Compute IoU between two boxes

CODE 05 / PYTHON GOOGLE COLAB

```python
def iou(box_a, box_b):
    xa=max(box_a[0],box_b[0]); ya=max(box_a[1],box_b[1])
    xb=min(box_a[2],box_b[2]); yb=min(box_a[3],box_b[3])
    intersection=max(0,xb-xa)*max(0,yb-ya)
    area_a=max(0,box_a[2]-box_a[0])*max(0,box_a[3]-box_a[1])
    area_b=max(0,box_b[2]-box_b[0])*max(0,box_b[3]-box_b[1])
    return float(intersection / max(area_a+area_b-intersection, 1e-8))

print('IoU(first proposal, first object):', round(iou(proposals[0],gt[0][0]),3)) if
    len(proposals) else None
```

Checkpoint 2 — Proposals

Check: Region proposal overlays are visible · IoU calculation produces a value between 0 and 1 · Students vary MAX_PROPOSALS and note latency

01. Why are there more proposals than actual objects?

02. What happens if a true object has no sufficiently overlapping proposal?

03. How would image texture or lighting affect Selective Search?

TASK 3 R-CNN Feature Extraction

Original R-CNN sends **each candidate crop separately** through a CNN. Here we use a frozen ImageNet ResNet18 feature extractor (512 dimensions) to make the exercise practical on Colab. This differs from the original R-CNN implementation, which used an AlexNet-style network followed by class-specific SVMs. For training, bounding boxes with **IoU ≥ 0.5** receive their matching object class, **IoU < 0.2** are background, and intermediate overlaps are ignored. Ground-truth boxes are also included **only in training**, ensuring positive examples for the educational classifier; they are never inserted into test proposals.

CODE 06 — Build labeled training proposals from ground truth

CODE 06 / PYTHON GOOGLE COLAB

```python
BACKGROUND = -1
TRAIN_MAX_NEG = 30  # negative regions sampled per training image

def label_proposal(box, gt):
    if not gt: return BACKGROUND
    matches=[(iou(box,gbox),cls) for gbox,cls in gt]
    score, cls=max(matches, key=lambda t:t[0])
    if score>=0.5: return cls
    if score<0.2: return BACKGROUND
    return None  # ignore ambiguous candidate

def build_training_rois(records):
    patches,labels=[],[]
    for rec in tqdm(records,desc='Preparing proposals'):
        image, gt=prepare_image(rec)
        regions=selective_proposals(image)
        # Training-only positive augmentation using *real ground-truth boxes*.
        for box,cls in gt:
            x1,y1,x2,y2=np.round(box).astype(int)
            if x2-x1>=8 and y2-y1>=8:
                patches.append(image[y1:y2,x1:x2]); labels.append(cls)
        neg=[]
        for box in regions:
            cls=label_proposal(box,gt)
```

CODE 06 / PYTHON / CONTINUED GOOGLE COLAB

```python
            if cls is None: continue
            x1,y1,x2,y2=np.round(box).astype(int)
            if x2-x1<8 or y2-y1<8: continue
            roi=image[y1:y2,x1:x2]
            if cls == BACKGROUND: neg.append(roi)
            else: patches.append(roi);labels.append(cls)
        random.shuffle(neg)
        for roi in neg[:TRAIN_MAX_NEG]: patches.append(roi);labels.append(BACKGROUND)
    return patches,np.array(labels,dtype=int)

train_patches,y_train=build_training_rois(train_records)
print('Training region labels:',dict(Counter(y_train)))
assert len(np.unique(y_train))>=2, 'Need positive classes and background; try a larger training subset.'
```

CODE 07 — Extract deep features from each region independently

CODE 07 / PYTHON GOOGLE COLAB

```python
from PIL import Image
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms

device=torch.device('cuda' if torch.cuda.is_available() else 'cpu')
weights=models.ResNet18_Weights.DEFAULT
backbone=models.resnet18(weights=weights)
backbone.fc=torch.nn.Identity()
backbone=backbone.to(device).eval()
preprocess=weights.transforms()

class ROIDataset(Dataset):
    def __init__(self,images): self.images=images
    def __len__(self): return len(self.images)
    def __getitem__(self,index):
        return preprocess(Image.fromarray(self.images[index]))

@torch.no_grad()
def extract_features(patches,batch_size=32):
    if not patches: return np.empty((0,512),dtype=np.float32)
    loader=DataLoader(ROIDataset(patches),batch_size=batch_size,shuffle=False,num_workers=0)
    result=[]
    for batch in tqdm(loader,desc='Independent CNN passes',leave=False):
        result.append(backbone(batch.to(device)).cpu().numpy())
    return np.concatenate(result,axis=0)

X_train=extract_features(train_patches)
print('Feature matrix:',X_train.shape)
```

Checkpoint 3 — Features

Check: Each region is processed separately · Feature matrix has one row per region · Feature vector dimension is recorded

01. Why is processing proposals independently expensive?

02. Why freeze the pretrained CNN in this lab?

03. How does feature dimensionality affect classifier training?

TASK 4 Train a Real SVM Classifier

Unlike the supplied handout's placeholder example, this lab never generates arbitrary training labels. The class targets come from the actual COCO boxes and each proposal's IoU. Train one linear SVM with classes **background**, **person**, **car**, and **dog** when represented in the subset. Decision scores are margins, **not calibrated probabilities**.

CODE 08 — Fit a linear SVM and check class distribution

CODE 08 / PYTHON GOOGLE COLAB

```python
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

svm=make_pipeline(StandardScaler(),
    LinearSVC(class_weight='balanced',max_iter=6000,random_state=SEED))
svm.fit(X_train,y_train)
print('Learned classes:',svm.named_steps['linearsvc'].classes_)
print('Training classification report (illustrative, NOT test accuracy):')
print(classification_report(y_train, svm.predict(X_train),zero_division=0))
```

Checkpoint 4 — SVM

Check: SVM trained on authentic IoU-derived labels · Present classes and their number of training regions · Students can explain margin scores versus probabilities

01. Why should random class labels never be used to demonstrate detector accuracy?
02. What does class weighting do?
03. Why is training accuracy not a valid estimate of detection performance?

TASK 5 Detection and Non-Maximum Suppression (NMS)

On unseen test images, proposals come exclusively from Selective Search. The pretrained CNN extracts their features, and the SVM provides a class and a margin. A margin threshold can filter weak detections. We apply **NMS independently per object class**; otherwise overlapping boxes from different classes could incorrectly suppress each other. This simplified lab does **not** include learned bounding-box regression.

CODE 09 — Run predictions and class-wise IoU-based NMS

CODE 09 / PYTHON GOOGLE COLAB

```python
def nms(boxes,scores,threshold=0.30):
    if len(boxes)==0: return []
    order=np.argsort(scores)[::-1]
    keep=[]
    while len(order):
        i=order[0]; keep.append(int(i))
        order=np.array([j for j in order[1:] if iou(boxes[i],boxes[j])<threshold],dtype=int)
    return keep

def detect(record, margin_threshold=0.0, nms_iou=0.30):
    image,gt=prepare_image(record)
    boxes=selective_proposals(image)
    patches=[]; valid=[]
    for box in boxes:
```

CODE 09 / PYTHON / CONTINUED GOOGLE COLAB

```python
        x1,y1,x2,y2=np.round(box).astype(int)
        if x2-x1>=8 and y2-y1>=8:
            patches.append(image[y1:y2,x1:x2]);valid.append(box)
    if not patches: return image,gt,[]
    X=extract_features(patches)
    classes=svm.named_steps['linearsvc'].classes_
    pred=svm.predict(X)
    margins=svm.decision_function(X)
    # Multiclass LinearSVC exposes one score per class; binary is handled separately.
    if margins.ndim==1:
        scores=np.where(pred==classes[1], margins, -margins)
    else:
        indices=np.array([np.where(classes==c)[0][0] for c in pred])
        scores=margins[np.arange(len(pred)),indices]
    det=[]
    for cls in classes:
        if cls==BACKGROUND: continue
        ids=[i for i in range(len(pred)) if pred[i]==cls and scores[i]>=margin_threshold]
        if not ids: continue
        b=np.array([valid[i] for i in ids]); s=np.array([scores[i] for i in ids])
        for j in nms(b,s,nms_iou):
            det.append({'box':b[j], 'class':int(cls), 'score':float(s[j])})
    return image,gt,det

def show_detections(record,threshold=0.0):
    image,gt,det=detect(record,margin_threshold=threshold)
    fig,ax=plt.subplots(figsize=(8,8));ax.imshow(image)
    for item in det:
        x1,y1,x2,y2=item['box']
        ax.add_patch(Rectangle((x1,y1),x2-x1,y2-y1,fill=False,edgecolor='red',linewidth=2))
        ax.text(x1,y1,f"{COCO_CLASSES[item['class']]}
    {item['score']:.2f}",color='yellow',fontsize=9)
    ax.set_title(f'R-CNN-style predictions ({len(det)} boxes after
    NMS)');ax.axis('off');plt.show()
    return det

predictions=show_detections(test_records[0])
```

Checkpoint 5 — Detection

Check: Predictions displayed on an unseen test image · SVM margins (not probabilities) labeled correctly · NMS applied separately to each class

01. Why must NMS use overlap between bounding boxes?

02. What changes when NMS IoU is reduced from 0.5 to 0.2?

03. Why may detections remain poor even if the SVM trains successfully?

TASK 6 Evaluate Real Detections

Measure **precision and recall at IoU ≥ 0.5** using authentic ground-truth boxes. We also compute **AP@0.5 per class** by sorting predictions by SVM score, matching each ground-truth box at most once, and integrating the precision envelope over recall. The mean of class AP values is called **mAP@0.5** here, **not COCO mAP@[.50:.95]**. Classes without ground-truth objects in the test subset are omitted from the mean. With so few images, scores are unstable and should not be interpreted as a benchmark.

CODE 10 — Calculate AP@0.5 using COCO ground-truth boxes

CODE 10 / PYTHON GOOGLE COLAB

```python
import pandas as pd

results=[]
for rec in tqdm(test_records,desc='Evaluating test images'):
    _,gt,det=detect(rec)
    results.append((gt,det))

def evaluate_class(results,cls,iou_threshold=.5):
    gt_by_image={i:[b for b,c in gt if c==cls] for i,(gt,_) in enumerate(results)}
    total_gt=sum(map(len,gt_by_image.values()))
    if total_gt==0: return None
    ranked=[]
    for image_id,(_,detections) in enumerate(results):
        ranked += [(d['score'],image_id,d['box']) for d in detections if d['class']==cls]
    ranked.sort(key=lambda a:a[0],reverse=True)
    matched={i:set() for i in gt_by_image}
    tp=[]; fp=[]
    for score,i,box in ranked:
        gt=gt_by_image[i]
        matches=[iou(box,g) if j not in matched[i] else -1 for j,g in enumerate(gt)]
        j=int(np.argmax(matches)) if matches else -1
        good=j>=0 and matches[j]>=iou_threshold
        if good: matched[i].add(j)
        tp.append(int(good)); fp.append(int(not good))
    if not tp: return {'class':COCO_CLASSES[cls],
    'gt':total_gt,'detections':0,'precision':0.,'recall':0.,'AP50':0.}
    tp=np.cumsum(tp);fp=np.cumsum(fp)
    recall=tp/total_gt;precision=tp/np.maximum(tp+fp,1)
    r=np.r_[0.,recall,1.];p=np.r_[0.,precision,0.]
    for k in range(len(p)-2,-1,-1):p[k]=max(p[k],p[k+1])
    changes=np.where(r[1:]!=r[:-1])[0]
    ap=float(np.sum((r[changes+1]-r[changes])*p[changes+1]))
    return {'class':COCO_CLASSES[cls], 'gt':total_gt,'detections':len(ranked),
            'precision':float(precision[-1]),'recall':float(recall[-1]),'AP50':ap}

rows=[evaluate_class(results,c) for c in COCO_CLASSES]
rows=[row for row in rows if row is not None]
summary=pd.DataFrame(rows)
print(summary.to_string(index=False))
print('mAP@0.5 (represented test classes):',round(float(summary['AP50'].mean()),3))
```

Checkpoint 6 — Evaluation

Check: Test predictions use no injected ground-truth candidate boxes · AP@0.5 is computed with one-to-one matching · Absent ground-truth classes are excluded from mAP

01. How does a localization error lower AP even if the class is correct?

02. Why is AP for a classification task different from object-detection AP?

03. How does a small, imbalanced test subset affect reported mAP?

TASK 7 Optional Backbone Variation and Reflection

Try another pretrained feature extractor (e.g., MobileNet or ResNet50), **re-extract all training and test proposal features**, and retrain the SVM before comparing results. Reusing a classifier trained on ResNet18 features with a different backbone is invalid. The principal educational trade-off is feature quality against computation time.

Final Checkpoint — Pipeline Understanding

Check: Show the seven main steps: labels, proposals, IoU assignment, CNN, SVM, NMS, AP · Summarize one correct detection and one missed detection · Report number of images, proposals and hardware

01. Why does R-CNN require many CNN evaluations per image?

02. What is the role of the SVM after feature extraction?

03. Which R-CNN components are trained in this laboratory?

04. Which element of the original R-CNN architecture is omitted?

05. Explain how Selective Search recall limits final detector recall.

Submission

Submit (1) the executed Colab notebook with outputs, (2) screenshots of COCO labels, proposal boxes and final predictions, (3) a table of per-class AP@0.5 and mean AP, (4) answers to all checkpoint questions, and (5) a short paragraph explaining limitations. Do not present results as a faithful reproduction of the original paper.